Software Requirements Specification (SRS)

Student Record Manager

Version: 1.0

Project Type: Python Mini Project 1.1
Frontend: HTML/CSS

Backend: Python
Storage: CSV/JSON File

Document Control
Prepared by: Md. Anjir Jaman
Date: 03-09-2026
Version: 1.0 – Initial draft

1. Table of Contents

1. Introduction

1.1 Purpose

The Student Record Manager is a web-based application designed to manage student information.

The application will provide a simple HTML-based interface through which users can add, view, search, update, and delete student records.

Student records will be stored in a local JSON file so that the data remains available after the application is closed.

1.2 Scope

The system will provide the following main functions:

- Add student records
- View all student records
- Search for students
- Update student information
- Delete student records
- Store student records in JSON
- Load existing records when the application starts
- Validate user input
- Display appropriate success and error messages

The front-end will be developed using basic HTML/CSS, while Python will handle the application logic and data processing.

1.3 Intended User

The system is intended for a user who needs to maintain a small collection of student records. The user does not need advanced technical knowledge to operate the application.

2. Overall Description

2.1 Product Perspective

The Student Record Manager will operate as a locally hosted Python web application, served using the Flask framework and accessed through a web browser at localhost. The system will use a local JSON file for data storage. No external database or internet connection is required.

Basic system flow:

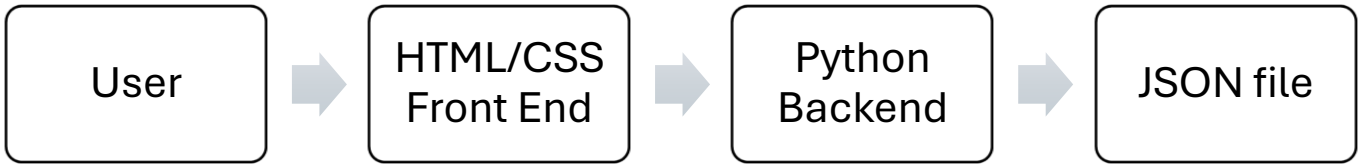

Figure 1: System Architecture

The frontend will collect user input and display information. The Python backend will process requests, perform validation, manage student records, and handle file operations. The CSV or JSON file will provide persistent storage.

2.2 Main Features

The system will provide the following functions

1. Dashboard/Home
2. Add Student
3. View Students
4. Search Student
5. Update Student
6. Delete Student

2.3 Operating Environment

The application will require:

1. A computer
2. Python
3. A modern web browser
4. A Python development environment such as VS Code or PyCharm
5. Local file storage
6. 6. The Flask micro-framework (Python), used to serve the web application

3. Functional Requirements

FR-01 Add Student

The system shall provide an HTML form for adding a new student.

The user shall enter the required student information.

The Python backend shall validate the submitted information.

If the information is valid, the system shall create and save the student record.

The system shall prevent duplicate Student IDs.

The system shall display a success message after a student is added successfully.

FR-02: View Students

The system shall provide a page displaying all stored student records.

Student records shall be displayed in a structured table.

The table shall show the relevant information for each student.

If no student records exist, the system shall display an appropriate message.

FR-03: Search Student

The system shall provide a search function.

The user shall be able to search for a student using the Student ID.

The backend shall search the stored records.

If a matching student is found, the system shall display the student's information.

If no matching student is found, the system shall display an appropriate message.

FR-04: Update Student

The system shall allow the user to update an existing student record.

The user shall identify the student using the Student ID.

The system shall display the student's existing information.

The user shall be able to modify the required information.

After submission, the backend shall validate and save the updated record.

The system shall display a confirmation message after a successful update.

FR-05: Delete Student

The system shall allow the user to delete an existing student record.

The user shall identify the student using the Student ID.

The system should request confirmation before deletion.

After confirmation, the system shall remove the student record from storage.

The system shall display a confirmation message after successful deletion.

FR-06: Save Student Records

The system shall store student records in a local JSON file.

The storage file shall be updated after adding, updating, or deleting a record.

Student data shall remain available after the application is closed.

FR-07: Load Student Records

The system shall load existing student records from the storage file when required.

If the storage file does not exist, the system shall create or initialize an empty record set.

FR-08: Input Validation

The system shall validate user input before storing or modifying records.

The system shall detect invalid or missing required information.

The system shall prevent duplicate Student IDs.

The system shall display clear error messages when invalid information is submitted.

The system shall validate that Email follows a standard email format (e.g., contains '@' and a domain).

The system shall validate that Phone contains only digits and is of a reasonable length (e.g., 10–14 digits).

FR-09: Navigation

The frontend shall provide navigation between the main application pages.

The user shall be able to move between:

- Dashboard
- Add Student
- View Students
- Search Student

- Update Student
- Delete Student

The user shall be able to return to the main dashboard from the other pages.

4. Data Requirements

Each student record shall contain:

Field	Description	Required
Student ID	Unique student identifier	Yes
Name	Student's full name	Yes
Department	Student's department	Yes
Email	Student's email address	Yes
Phone	Student's phone number	Optional

5. User Interface Requirements

The application shall use a basic HTML/CSS frontend.

5.1 Dashboard

The dashboard shall provide access to the main functions of the application.

5.2 Add Student Page

The page shall contain input fields for:

- Student ID
- Name
- Department
- Email
- Phone

It shall provide an **Add Student** button.

5.3 Student List Page

The page shall display student records in a table.

Example:

ID	Name	Department	Email	Phone	Actions
1001	MD Jaman	CSE	jaman@email.com	017...	Edit/Delete

5.4 Search Page

The page shall provide a Student ID input field and a search button.

The matching student's information shall be displayed when found.

5.5 Update Page

The page shall allow the user to select a student and modify the student's information.

5.6 Delete Function

The interface shall provide a delete option for student records.

A confirmation should be displayed before the record is permanently deleted.

6. Backend Requirements

The Python backend shall:

- Receive data from the HTML frontend
- Validate submitted information
- Add student records
- Retrieve student records
- Search records
- Update records
- Delete records
- Read from the storage file
- Write changes to the storage file
- Return appropriate results and messages to the frontend

The backend should keep the application logic separate from the HTML presentation where practical.

7. Data Storage Requirements

The system shall use either JSON for local data storage.

Example JSON:

{ "1001": {"name": "John Doe", "department": "CSE", "email": "john@email.com", "phone": "01700000000"},

"1002": {"name": "Jane Smith", "department": "EEE", "email": "jane@email.com", "phone": "01800000000"} } ` ` ` "

The system shall support:

- Creating records
- Reading records
- Updating records
- Deleting records
- Persistent storage

8. Non-Functional Requirements

NFR-01: Usability

The interface should be simple, clear, and easy to navigate.

NFR-02: Performance

Normal operations should be completed within a reasonable time for the expected small number of records.

NFR-03: Reliability

The system should correctly save and retrieve student information during normal operation.

NFR-04: Maintainability

The Python code should be organized clearly so that the application can be modified or extended easily.

NFR-05: Compatibility

The frontend should work with a modern web browser.

NFR-06: Data Persistence

Student information shall remain available after the application is closed and reopened.

9. Error Handling

The system shall handle common errors without crashing.

The system shall provide appropriate messages for:

- Missing required fields
- Invalid input
- Duplicate Student ID
- Student not found
- Invalid search
- Updating a non-existing student
- Deleting a non-existing student
- Missing storage file
- File read/write errors

After an error, the user should be able to return to the appropriate page and try again.

10. System Constraints

- The application will be developed using Python.
- The frontend will use basic HTML/CSS.
- Data will be stored locally using CSV or JSON.
- No internet connection is required for normal operation.
- No external database is required.
- The system is intended for a small dataset.
- The application will run locally through a web browser.
- The project will remain within the scope of a Python mini project.

11. Assumptions

- Python is installed on the development computer.
- A modern web browser is available.
- The user has permission to read and write files in the project directory.

- Student IDs are unique.
- The user provides valid student information.
- The storage file is accessible to the application.

12. Acceptance Criteria

The project will be considered complete when:

- The HTML frontend loads successfully.
- The user can navigate between application pages.
- A student can be added through the frontend.
- Student records can be viewed in a table.
- A student can be searched using Student ID.
- Existing student information can be updated.
- A student can be deleted.
- Records are saved to CSV or JSON.
- Existing records are loaded correctly.
- Duplicate Student IDs are prevented.
- Invalid input is handled correctly.
- Appropriate success and error messages are displayed.
- The application does not crash during normal use.
- All major functions pass their test cases.

13. Project Schedule and Milestones

The project is scheduled to be completed in a single working day (10:00 AM – 6:00 PM), broken down by task as shown in Figure 2. The schedule follows the SDLC sequence defined in this document: requirements review, design, backend implementation, frontend implementation, testing, and reporting.

Task	10am	11am	12pm	1pm	2pm	3pm	4pm	5pm	6pm
Review SRS, set up Flask project									
Design routes and JSON schema									
Backend: CRUD functions									
Frontend: HTML & CSS									
Integration and testing									
Write Report									

Figure 2: Development schedule

14. Requirement Summary

ID Requirement

FR-01	Add student
FR-02	View students
FR-03	Search student
FR-04	Update student
FR-05	Delete student
FR-06	Save records
FR-07	Load records
FR-08	Validate input

FR-09	Navigation
NFR-01	Usability
NFR-02	Performance
NFR-03	Reliability
NFR-04	Maintainability
NFR-05	Browser compatibility
NFR-06	Data persistence